

ESP32 2 — I²C OLED Display

Tutorial Group: ESP32

Goal

In this tutorial, you will learn how to connect an OLED display to the ESP32 using I²C. You will use the ESP32's analog input to read the variable resistor from the previous tutorials, convert the ADC reading into an approximate voltage, and display the results on the OLED.

The display will also show your team's name. This tutorial combines analog input, ADC values, voltage conversion, const variables, and a new hardware interface: I²C.

What You Need

- ESP32-WROOM-32
- USB cable
- Breadboard and jumper wires
- Variable resistor/potentiometer from the Mega 2560 evaluation kit
- SSD1306, 128x64, I²C OLED display with a supply-voltage range compatible with both the Arduino Mega 2560 and ESP32-WROOM-32

Part 1 — What Is I²C?

I²C is a communication interface commonly used to connect sensors, displays, and other devices to a microcontroller. Unlike a simple digital output, I²C allows the ESP32 to exchange data with another device.

I²C normally uses two signal lines:

- SDA — Serial Data
- SCL — Serial Clock

In addition to SDA and SCL, the devices need a common ground and power. The OLED therefore requires only a small number of connections.

Part 2 — ESP32 I²C Connections

In the *Arduino 5 — I²C MPU-6050 Accelerometer & Gyroscope* tutorial, we learned about I²C and how to use it on the Mega 2560. In this tutorial, we learn about the differences between the Mega 2560 I²C and the ESP32 I²C.

For this tutorial, we will use the ESP32's standard I²C pins:

OLED	ESP32
VCC	3.30v
GND	GND
SDA	GPIO 21
SCL	GPIO 22

The standard I²C pins used in this tutorial are:

SDA = GPIO 21

```
SCL = GPIO 22
```

The ESP32 operates at 3.30v logic. Do not assume that connections used for the Arduino Mega 2560 can simply be transferred to the ESP32.

Part 3 — Connect the OLED

Connect the OLED to the ESP32 according to the table above. The OLED does not use a normal digitalWrite() connection. Instead, the ESP32 sends commands and display data through the I²C interface.

Part 4 — Install the OLED Libraries

Use the Adafruit SSD1306 library and the Adafruit GFX library.

- Open the Arduino IDE Library Manager.
- Search for Adafruit SSD1306 and install the library from Adafruit.
- Install Adafruit GFX Library if it is not installed automatically.

These libraries provide the functions needed to draw text and graphics on the OLED.

Part 5 — Define the Team Name

At the top of the program, create a constant containing your team name:

```
const char teamName[] = "Team 42";
```

Change the value to your team's name. The team name is defined once and can then be used throughout the program as teamName. A team name is a sequence of characters rather than a number. The const keyword indicates that the program will not change it while running.

Note: at text size 2, the 128-pixel-wide display can show up to 10 characters on one line. For this example, use a team name of 10 characters or fewer. Later, you will experiment with different text sizes.

Part 6 — Create the OLED Program

Start with the following program and upload it to the ESP32:

The example below uses I²C address 0x3C. Some SSD1306 I²C displays use a different address. If the display is not detected, check the display module's documentation or determine its I²C address before changing the wiring.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

const char teamName[] = "Team 42";

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup()
{
  Serial.begin(115200);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    Serial.println("OLED not found");
  }
}
```

```

    while (1);
}

display.clearDisplay();

display.setTextColor(SSD1306_WHITE);
display.setTextSize(2);
display.setCursor(0, 0);

display.println(teamName);
display.println("DONT PANIC");

display.display();
}

void loop()
{
}

```

Your team's name should appear on the OLED.

Part 7 — Understanding the OLED Code

The program introduces several new functions.

```

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

```

Wire.h provides I²C functionality. Adafruit_GFX provides general graphics functions. Adafruit_SSD1306 provides functions for the SSD1306 OLED controller.

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

You may notice that the program uses Wire without declaring a variable named Wire. Wire is an object that is already declared by the Wire library. The statement `#include <Wire.h>` makes the library's declarations available to your program, including the pre-created Wire object. That is why the program can call functions such as `Wire.begin()` without first writing a declaration such as `Wire wire;`. In this tutorial, Wire is the library-provided object used to access the ESP32's I²C interface.

This creates the display object that the program uses to communicate with the OLED.

```

display.clearDisplay();
display.setTextColor(SSD1306_WHITE);
display.setTextSize(2);
display.setCursor(0, 0);

```

These functions clear the display buffer, select the display color, set the text size, and position the cursor. The OLED coordinate system starts at (0,0) in the upper-left corner. The '2' of `setFontSize` specifies a 12x16 pixel character, so 10 characters can fit on the 128 pixel display.

```
display.println(teamName);  
display.println("DONT PANIC");  
display.display();
```

The first line places the team name in the display buffer, and the second line is a simple *Hitchhiker's Guide to the Galaxy* Easter egg¹ (*if you know, you know*), while `display()` transfers the buffer to the actual OLED.

This distinction of `print`, `println`, and `display` becomes useful when building more complex displays.

Part 8 — Add the Analog Input

Connect the variable resistor from the previous tutorials. Use 3.30v and GND on the two outside terminals and connect the wiper to GPIO 34.

```
const int analogPin = 34;
```

GPIO 34 is an ADC-capable input on the ESP32. Unlike the Arduino Mega 2560, ESP32 pin numbering and ADC behavior are different, so always check the ESP32 documentation when selecting a pin.

Part 9 — Read the ADC

Inside `loop()`, read the analog input:

```
int adcValue = analogRead(analogPin);
```

The ESP32 ADC is treated as a 12-bit ADC in this tutorial. Twelve bits provide $2^{12} = 4096$ possible codes, numbered from 0 through 4095. Therefore, a reading of 0 represents the low end of the range and 4095 represents the high end of the range.

Part 10 — Convert the ADC Value to Voltage

For this tutorial, use the following simplified approximation for the ADC-to-voltage calculation. It assumes 0 corresponds to 0.00v and 4095 corresponds approximately to 3.30v. Dividing by 4095 is why the equation uses 4095 rather than 4096. This is useful for illustrating the relationship between ADC values and voltage, but it is not a calibrated ESP32 voltage measurement:

```
float voltage = adcValue * 3.3 / 4095.0;
```

The voltage is stored as a float because the result can contain a fractional value. For example, an ADC reading near 2048 is about halfway through the 0–4095 range, so the simplified equation gives approximately 1.65v.

For a more accurate ESP32 voltage measurement, the ADC attenuation and calibration must be taken into account. This tutorial intentionally uses the simplified equation so the focus remains on the ADC-to-voltage relationship.

Note: 3.30v is used as an approximate reference in this tutorial. The actual ESP32 ADC response depends on the ADC configuration, attenuation, and calibration, so this equation should not be treated as a precision voltage measurement.

¹ The **five** books in the *Hitchhiker's Guide to the Galaxy* **trilogy** make great reading to get your mind off a stressful exam or even a competition. If you have not already read them, put them high on your list, and if you have read them, read them again (and again).

Part 11 — Display the Analog Information

Replace the empty loop() with:

```
void loop()
{
  int adcValue = analogRead(analogPin);

  float voltage = adcValue * 3.3 / 4095.0;

  display.clearDisplay();

  display.setTextColor(SSD1306_WHITE);

  display.setTextSize(2);
  display.setCursor(0, 0);
  display.println(teamName);

  display.setTextSize(1);
  display.setCursor(0, 25);
  display.print("ADC: ");
  display.println(adcValue);

  display.setCursor(0, 40);
  display.print("Voltage: ");
  display.print(voltage, 2);
  display.println("v");

  display.display();

  delay(100);
}
```

Upload the program and turn the potentiometer. The ADC value and calculated voltage should change on the OLED.

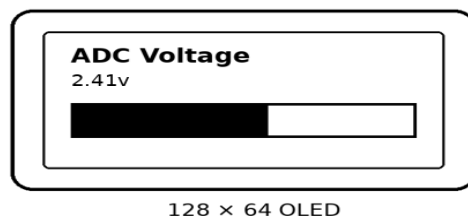
Part 12 — Add a Simple Bar Graph

The OLED can display graphics as well as text. Add the following after the voltage display:

```
int barWidth = map(adcValue, 0, 4095, 0, 120);

display.drawRect(4, 55, 120, 7, SSD1306_WHITE);
display.fillRect(4, 55, barWidth, 7, SSD1306_WHITE);
```

map() converts the ADC range of 0–4095 into a display width of 0–120 pixels. The two drawing functions create an outline and a filled bar. As the potentiometer is turned, the bar grows and shrinks.



Part 13 — Complete Program

Your completed program should look similar to this:

```

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

const char teamName[] = "Team 42";
const int analogPin = 34;

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup()
{
    Serial.begin(115200);

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
    {
        Serial.println("OLED not found");
        while (1);
    }
}

void loop()
{
    int adcValue = analogRead(analogPin);

    float voltage = adcValue * 3.3 / 4095.0;

    int barWidth = map(adcValue, 0, 4095, 0, 120);

    display.clearDisplay();

    display.setTextColor(SSD1306_WHITE);

    display.setTextSize(2);
    display.setCursor(0, 0);
    display.println(teamName);

    display.setTextSize(1);

    display.setCursor(0, 25);
    display.print("ADC: ");
    display.println(adcValue);

    display.setCursor(0, 40);
    display.print("Voltage: ");
    display.print(voltage, 2);
    display.println("v");

    display.drawRect(4, 55, 120, 7, SSD1306_WHITE);
    display.fillRect(4, 55, barWidth, 7, SSD1306_WHITE);

    display.display();

    delay(100);
}

```

Part 14 — Experiment With the Display

Experiment with the OLED after the basic program works.

- Change `display.setTextSize(2)` to `display.setTextSize(1)`.
- Change the cursor position and move the team name around the display.
- Experiment with the size and position of the bar graph.
- Try different combinations of text sizes and positions without obscuring the other information.

The purpose is to become comfortable with the idea that an OLED is a small graphics display, not simply a text display.

What You Should Understand

- What I²C is used for.
- The purpose of SDA and SCL.
- Which ESP32 GPIOs are used for I²C in this tutorial.
- How an OLED is connected to the ESP32.
- How an Arduino library provides an interface to a hardware device.
- How to display text and position it using pixel coordinates.
- How the OLED display buffer works.
- How to read an ESP32 ADC input.
- How to convert an ADC value into an approximate voltage.
- Why a float is appropriate for the voltage.
- How to draw simple graphics on the OLED.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can connect an I²C OLED to the ESP32.
- ☐ I know what SDA and SCL are.
- ☐ I understand that I²C is a communication interface rather than a simple GPIO output.
- ☐ I can display text on the OLED.
- ☐ I can change text size and position.
- ☐ I can read an analog input on the ESP32.
- ☐ I can convert the ADC reading to an approximate voltage.
- ☐ I understand why the voltage is stored as a float.
- ☐ I can display both numerical information and simple graphics on the OLED.
- ☐ I can change the team name by modifying one `const char` declaration at the top of the program.

Once this tutorial is complete, you should be comfortable using an I²C OLED with the ESP32 and combining sensor input, calculations, text, and simple graphics in a single program.